

Documentación Técnica: Modelos Fundacionales para Series Temporales e Infraestructura GPU

Trabajo de Fin de Grado — Predicción de Fatiga con FATIGUESet

Emilio Gullón

Julio 2026

Resumen

Este documento recoge la implementación, despliegue y evaluación experimental de tres modelos fundacionales para series temporales — MOMENT (encoder-only), CHRONOS (decoder-only probabilístico) y TIMESFM (decoder-only con parches) — aplicados a la predicción de fatiga física y mental a partir de señales fisiológicas del dataset FATIGUESet. Se describe en detalle la librería `fatigueset.models.foundation`, el módulo de optimización de hiperparámetros con Optuna (`fatigueset.models.optimizers`), la infraestructura de ejecución en un clúster GPU universitario (UGR), y los resultados comparativos frente a modelos clásicos de Machine Learning y redes de Deep Learning diseñadas a medida. Adicionalmente, se documentan los problemas técnicos encontrados (errores de memoria GPU, límites de tiempo en SLURM) y las soluciones aplicadas.

Índice

1. Introducción	3
1.1. Contexto del TFG y FatigueSet	3
1.2. Motivación: modelos fundacionales para series temporales	3
1.3. Objetivos de esta fase experimental	3
2. Marco Teórico: Modelos Fundacionales para Series Temporales	4
2.1. Taxonomía de arquitecturas	4
2.2. MOMENT: arquitectura, parches y preentrenamiento	4
2.3. Chronos: tokenización por quantile binning y T5	5
2.4. TimesFM: parches de longitud variable y decoder patched	6
3. Implementación de la Librería <code>fatigueset.models.foundation</code>	6
3.1. <code>MOMENTFatigueRegressor</code> : diseño y forward pass	6
3.2. <code>ChronosZeroShotEvaluator</code> : sonda lineal sobre features zero-shot	7
3.3. <code>TimesFMZeroShotEvaluator</code> : cuantiles y predicción por canal	7
3.4. Métricas probabilísticas: CRPS y cobertura	8
4. Módulo de Optimizadores (<code>fatigueset.models.optimizers</code>)	8
4.1. Arquitectura del módulo	8
4.2. Búsqueda condicional con Optuna	9
4.3. Espacios de búsqueda por familia de modelo	9

5. Infraestructura de Computación en Servidor GPU	10
5.1. Arquitectura del clúster UGR	10
5.2. Entorno Conda remoto	11
5.3. Scripts SLURM de ejecución	11
5.4. Pipeline de despliegue automático	11
5.5. Monitorización remota de jobs	12
6. Resultados Experimentales	12
6.1. Tabla comparativa global	12
6.2. MOMENT: resultados por fold	13
6.3. Chronos: métricas zero-shot (modelo tiny)	14
6.4. TimesFM: métricas puntuales y probabilísticas	14
6.5. Optuna: resultados del experimento básico	15
6.6. Discusión y análisis comparativo	15
7. Problemas Técnicos y Soluciones	16
7.1. CUDA Out of Memory: diagnóstico y corrección	16
7.2. Exceso de tiempo en SLURM (Optuna)	17
8. Conclusiones y Trabajo Futuro	17
8.1. Comparativa final de paradigmas	17
8.2. Tareas pendientes	18

1. Introducción

1.1. Contexto del TFG y FatigueSet

Este Trabajo de Fin de Grado aborda la predicción cuantitativa de fatiga física y mental a partir de señales fisiológicas multicanal del dataset FATIGUESet [1]. El dataset contiene registros de 12 participantes monitorizados con dispositivos portátiles (Empatica E4 y Zephyr BioHarness) durante tareas cognitivas y físicas, proporcionando 23 canales fisiológicos (acelerómetros, electrodermal, temperatura, ritmo cardíaco, respiración, etc.) junto con puntuaciones subjetivas de fatiga en escala 0–100.

El pipeline experimental se estructura en cuatro fases progresivas de complejidad:

1. **Modelos clásicos de Machine Learning:** Random Forest, KNN, Lasso, Ridge, SVM, Decision Tree, ElasticNet y Regresión Lineal.
2. **Modelos de Deep Learning diseñados a medida:** LSTM, GRU, CNN-LSTM, TCN, Transformer, PatchTST y xLSTM.
3. **Optimización de hiperparámetros con Optuna:** Búsqueda bayesiana con TPE (Tree-structured Parzen Estimator) y optimizadores personalizados.
4. **Modelos fundacionales preentrenados:** MOMENT, CHRONOS y TIMESFM.

1.2. Motivación: modelos fundacionales para series temporales

Los modelos fundacionales (*Foundation Models*) representan un cambio de paradigma en el análisis de series temporales. Frente al enfoque tradicional de diseñar y entrenar un modelo específico para cada tarea, estos modelos se preentrenan sobre grandes corpora de series temporales diversas y posteriormente se adaptan a tareas downstream con mínima supervisión.

El survey de Kottapalli et al. [2] identifica tres paradigmas de adaptación:

- **Zero-shot:** el modelo preentrenado se aplica directamente sin ningún reentrenamiento.
- **Fine-tuning:** se reentrenan las últimas capas o una cabeza de tarea ligera.
- **Prompting / In-context learning:** se proporcionan ejemplos de demostración al modelo.

En este TFG se aplican los tres modelos fundacionales más relevantes identificados en el survey, evaluando su capacidad como extractores de características universales para la predicción de fatiga en un dominio no visto durante su preentrenamiento.

1.3. Objetivos de esta fase experimental

Los objetivos específicos de la fase de modelos fundacionales son:

- (a) Implementar adaptadores para MOMENT, CHRONOS y TIMESFM dentro de la librería modular `fatigueset`.
- (b) Evaluar el rendimiento de cada modelo en predicción de fatiga física y mental usando validación cruzada GroupKFold por participante.

- (c) Comparar métricas deterministas (MAE, RMSE, R^2) y probabilísticas (CRPS, cobertura del intervalo de predicción al 90 %).
- (d) Desplegar y ejecutar los experimentos a escala completa en un clúster GPU universitario.

2. Marco Teórico: Modelos Fundacionales para Series Temporales

2.1. Taxonomía de arquitecturas

Siguiendo la taxonomía del survey de Kottapalli et al. [2], los modelos fundacionales para series temporales se clasifican según su arquitectura base:

- **Encoder-only**: procesan la secuencia completa bidireccionalmente para producir representaciones contextualizadas. Ejemplo: MOMENT.
- **Decoder-only**: procesan la secuencia de forma autorregresiva (causal), prediciendo el siguiente token/valor. Ejemplos: CHRONOS, TIMESFM.
- **Encoder-Decoder**: combinan ambos enfoques. Ejemplo: Lag-Llama (no implementado en este TFG).

La Tabla 1 resume la taxonomía de los modelos implementados.

Cuadro 1: Taxonomía de los modelos fundacionales implementados en el TFG.

Modelo	Tipo	Patch-based	Multivariado	Probabilístico	Paradigma
MOMENT	Encoder-only	Sí	Sí	No	Fine-tuning cabeza
CHRONOS	Decoder-only	No	No	Sí	Zero-shot + probe
TIMESFM	Decoder-only	Sí	No	Sí	Zero-shot + probe

2.2. MOMENT: arquitectura, parches y preentrenamiento

MOMENT (*A Family of Open Time-series Foundation Models*) [3] es un modelo encoder-only basado en Transformer preentrenado mediante reconstrucción de parches enmascarados (*Masked Patch Reconstruction*), análogo al preentrenamiento de BERT en NLP pero adaptado a series temporales.

Arquitectura. La secuencia temporal de entrada $\mathbf{x} \in \mathbb{R}^{T \times C}$ (con T pasos temporales y C canales) se segmenta en *parches* no superpuestos de longitud fija P . Cada parche se proyecta a un espacio de dimensión d_{model} mediante una capa lineal, produciendo una secuencia de tokens:

$$\mathbf{z}_i = W_{\text{patch}} \cdot \mathbf{x}_{[iP:(i+1)P]} + \mathbf{b}_{\text{patch}}, \quad i = 0, 1, \dots, \lfloor T/P \rfloor - 1$$

Los tokens se procesan por L capas de Transformer encoder (auto-atención multi-cabeza + MLP feed-forward), produciendo representaciones contextualizadas $\mathbf{h}_i$ para cada parche.

Preentrenamiento. MOMENT se preentrena en el corpus *Time Series Pile*, una colección de más de 1 billón de observaciones de series temporales de dominios diversos (finanzas, salud, clima, energía). El objetivo de preentrenamiento es la reconstrucción de parches enmascarados: se enmascara un porcentaje aleatorio de parches y el modelo debe reconstruirlos a partir del contexto.

Variantes. Se implementan dos variantes:

- **MOMENT-1-small:** 40M parámetros (pruebas locales).
- **MOMENT-1-large:** 385M parámetros (experimento en servidor, 341.250.570 parámetros totales).

Adaptación a FatigueSet. Para adaptar MOMENT a la predicción de fatiga, se congela el backbone preentrenado y se añade una cabeza de regresión lineal $\text{Linear}(d_{\text{model}}, 2)$ que mapea la representación agregada (media de los embeddings de todos los parches) a las dos dimensiones de fatiga. Solo se entrenan los 2.050 parámetros de la cabeza (0.00 % del total), lo que reduce drásticamente el riesgo de sobreajuste en el pequeño dataset de FATIGUSET.

2.3. Chronos: tokenización por quantile binning y T5

CHRONOS [4] es un framework probabilístico decoder-only para series temporales que reutiliza la arquitectura T5 (*Text-to-Text Transfer Transformer*) de Google, adaptándola al dominio temporal mediante una estrategia de tokenización numérica.

Tokenización. Los valores continuos de la serie temporal se discretizan en *bins* mediante *quantile binning*: se calculan los cuantiles empíricos del contexto y cada valor se asigna al bin correspondiente. Esta estrategia permite reutilizar el vocabulario discreto del T5 sin modificar la arquitectura.

Preentrenamiento. CHRONOS se preentrena con un objetivo de lenguaje estándar (predicción del siguiente token) sobre el corpus TSMixup, una mezcla de series temporales reales y sintéticas generadas por Gaussian Processes.

Variantes.

- **chronos-t5-tiny:** 8M parámetros (pruebas locales).
- **chronos-t5-large:** 710M parámetros (experimento en servidor).

Adaptación a FatigueSet. Dado que CHRONOS está preentrenado exclusivamente para *forecasting* univariado, la adaptación a regresión multidimensional de fatiga se realiza mediante un enfoque de **linear probe** (sonda lineal):

1. Cada uno de los 23 canales fisiológicos se trata como una serie temporal univariada independiente.
2. CHRONOS predice la distribución del siguiente paso para cada canal, obteniéndose la mediana como característica.

3. Las 23 medianas se concatenan como vector de características y se alimentan a una regresión Ridge que mapea al espacio de fatiga bidimensional.

2.4. TimesFM: parches de longitud variable y decoder patched

TIMESFM 2.5 (*Time Series Foundation Model*) [5] es un modelo decoder-only de Google Research basado en parches de longitud variable. A diferencia de MOMENT, que usa parches de longitud fija, TIMESFM utiliza un mecanismo de *input patching* con múltiples resoluciones que permite procesar series temporales de diferentes frecuencias sin re-muestreo.

Arquitectura. TIMESFM 2.5 utiliza un decoder Transformer con 200M de parámetros, entrenado sobre 100 billones de observaciones de series temporales reales y sintéticas. Su diseño clave es la predicción por parches con tamaño de salida variable (*output patch size*), lo que lo hace extremadamente eficiente en inferencia.

Adaptación a FatigueSet. La adaptación sigue el mismo paradigma de channel independence y linear probe que CHRONOS:

1. Los 23 canales se procesan independientemente por TIMESFM.
2. El modelo genera predicciones puntuales y distribuciones de cuantiles (10 cuantiles predefinidos) por canal.
3. Las predicciones puntuales se concatenan como vector de 23 características y se alimentan a una regresión Ridge.
4. Los cuantiles se usan para calcular métricas probabilísticas (CRPS, cobertura).

3. Implementación de la Librería `fatigueset.models.foundation`

El módulo `foundation.py` (924 líneas, 34.6 KB) implementa toda la lógica de adaptación de los tres modelos fundacionales. Se estructura en cinco componentes principales.

3.1. MOMENTFatigueRegressor: diseño y forward pass

La clase `MOMENTFatigueRegressor` hereda de `torch.nn.Module` y encapsula el backbone MOMENT con una cabeza de regresión personalizada.

```

1 class MOMENTFatigueRegressor(nn.Module):
2     def forward(self, x: torch.Tensor) -> torch.Tensor:
3         # Transponer: (B, seq_len, n_channels) -> (B, n_channels
4         #           , seq_len)
5         x_moment = x.transpose(1, 2).contiguous()
6
7         # Embeddings preentrenados (backbone congelado)
8         outputs = self._backbone.embed(x_enc=x_moment, reduction
9         = "mean")
10        pooled = outputs.embeddings # (B, d_model)

```

```

10         # Cabeza de regresion entrenada
11         return self.regression_head(self.dropout_layer(pooled))

```

Listing 1: Paso forward de MOMENTFatigueRegressor (simplificado).

Diseño de la cabeza. La cabeza de regresión consiste en:

- `Dropout(p=0.1)`: regularización para evitar sobreajuste.
- `Linear( $d_{\text{model}}$ , 2)`: proyección lineal al espacio de fatiga bidimensional, con inicialización Xavier uniforme para los pesos y ceros para los sesgos.

Transposición de ejes. MOMENT espera la entrada en formato (`batch`, `n_channels`, `seq_len`), mientras que el resto de modelos de FATIGUSET usan (`batch`, `seq_len`, `n_channels`). El método `forward` realiza la transposición internamente, garantizando la compatibilidad sin modificar el pipeline de datos.

Carga perezosa del backbone. El backbone se carga de forma perezosa (*lazy loading*): si se proporciona un backbone pre-cargado en el constructor (parámetro `backbone`), se reutiliza directamente; en caso contrario, se carga automáticamente en el primer `forward` pass mediante `MOMENTPipeline.from_pretrained`.

3.2. ChronosZeroShotEvaluator: sonda lineal sobre features zero-shot

La clase `ChronosZeroShotEvaluator` implementa la evaluación zero-shot de CHRONOS con tres métodos principales:

1. `extract_features(X)`: itera sobre los 23 canales, llama a `predict_quantiles` de CHRONOS para cada canal, y extrae la mediana (cuantil 0.5) del siguiente paso como característica. Resultado: matriz (N , 23).
2. `fit_probe(X_train, y_train, X_val)`: entrena una regresión Ridge ($\alpha = 1,0$) sobre las 23 características extraídas y predice sobre el conjunto de validación.
3. `predict_quantiles_for_crps(X)`: predice 9 cuantiles (0.1 a 0.9) por canal para el cálculo de métricas probabilísticas.

3.3. TimesFMZeroShotEvaluator: cuantiles y predicción por canal

La clase `TimesFMZeroShotEvaluator` sigue el mismo paradigma pero aprovecha la API nativa de TIMESFM 2.5:

```

1 def extract_features(self, X, batch_size=32):
2     N, seq_len, n_channels = X.shape
3     point_preds = np.zeros((N, n_channels))
4     quantile_preds = np.zeros((N, n_channels, 10))
5
6     for start in range(0, N, batch_size):
7         batch = X[start:end]

```

```

8         flat_inputs = [batch[i, :, ch]
9                         for i in range(B) for ch in range(
10                             n_channels)]
11
12         point_pred, quant_pred = self._model.forecast(
13             horizon=1, inputs=flat_inputs)
14         # Des-aplanar resultados...
15     return point_preds, quantile_preds

```

Listing 2: Extracción de características con TimesFM (simplificado).

La diferencia clave con CHRONOS es que TIMESFM genera directamente 10 cuantiles predefinidos en su salida nativa, sin necesidad de muestreo de Monte Carlo.

3.4. Métricas probabilísticas: CRPS y cobertura

Se implementan dos métricas probabilísticas estándar:

CRPS (Continuous Ranked Probability Score). Bajo el supuesto de distribución Normal, el CRPS se calcula analíticamente como [6]:

$$\text{CRPS}(\mu, \sigma, y) = \sigma \left[z(2\Phi(z) - 1) + 2\phi(z) - \frac{1}{\sqrt{\pi}} \right], \quad z = \frac{y - \mu}{\sigma}$$

donde $\Phi(\cdot)$ y $\phi(\cdot)$ son la función de distribución acumulada y la densidad de la Normal estándar, respectivamente.

El CRPS generaliza el MAE al espacio probabilístico: para predicciones deterministas ($\sigma \rightarrow 0$), el CRPS degenera al MAE estándar.

Cobertura del intervalo de predicción. La cobertura empírica mide la fracción de observaciones reales que caen dentro del intervalo de predicción:

$$\text{Coverage}_{1-\alpha} = \frac{1}{N} \sum_{i=1}^N \mathbf{1} \left[q_{\alpha/2}^{(i)} \leq y_i \leq q_{1-\alpha/2}^{(i)} \right]$$

Un intervalo de predicción del 90 % bien calibrado debería tener una cobertura empírica cercana a 0.90.

4. Módulo de Optimizadores (fatigueset.models.optimizers)

El módulo `optimizers.py` (477 líneas, 17.4 KB) extiende la búsqueda de hiperparámetros con Optuna al incluir la selección del **optimizador como hiperparámetro**.

4.1. Arquitectura del módulo

El módulo se estructura en cuatro componentes:

1. **build_optimizer:** Función de fábrica que instancia un optimizador de PyTorch a partir de su nombre y parámetros. Soporta Adam, AdamW, RMSprop y SGD (con momentum de Nesterov).

2. `sample_optimizer_params`: Define el espacio de búsqueda condicional del optimizador en Optuna. Primero se selecciona el tipo de optimizador como variable categórica, y luego se activan únicamente los hiperparámetros relevantes:
 - **Todos**: learning rate (10^{-5} a 10^{-2} , escala log) y weight decay (10^{-7} a 10^{-3} , escala log).
 - **SGD solamente**: momentum (0.7 a 0.99).
 - **RMSprop solamente**: alpha (0.9 a 0.999).
3. **Funciones `sample_*` por familia**: Una función de muestreo por cada familia de modelos (LSTM, GRU, CNN-LSTM, TCN, Transformer, PatchTST, xLSTM, Random Forest) que define el espacio de búsqueda completo de la arquitectura *incluyendo* el optimizador.
4. `sample_model_hyperparams`: Función unificada que permite la búsqueda jerárquica y condicional entre familias de modelos en un único estudio de Optuna.

4.2. Búsqueda condicional con Optuna

La búsqueda condicional es especialmente importante para evitar sugerir hiperparámetros irrelevantes (p. ej., momentum para Adam), lo que reduce la dimensionalidad efectiva del espacio de búsqueda y mejora la eficiencia del sampler TPE.

```

1 def sample_optimizer_params(trial, prefix):
2     optimizer_name = trial.suggest_categorical(
3         f"{prefix}_optimizer", ["Adam", "AdamW", "RMSprop", "SGD"]
4     )
5     lr = trial.suggest_float(f"{prefix}_lr", 1e-5, 1e-2, log=True)
6     weight_decay = trial.suggest_float(
7         f"{prefix}_weight_decay", 1e-7, 1e-3, log=True)
8
9     params = {"optimizer": optimizer_name, "lr": lr,
10              "weight_decay": weight_decay}
11
12     if optimizer_name == "SGD":
13         params["momentum"] = trial.suggest_float(
14             f"{prefix}_momentum", 0.7, 0.99)
15     elif optimizer_name == "RMSprop":
16         params["alpha"] = trial.suggest_float(
17             f"{prefix}_alpha", 0.9, 0.999)
18     return params

```

Listing 3: Búsqueda condicional de optimizador en Optuna.

4.3. Espacios de búsqueda por familia de modelo

La Tabla 2 resume los hiperparámetros buscados por Optuna para cada familia de modelo.

Cuadro 2: Resumen de hiperparámetros explorados por Optuna por familia de modelo.

Familia	Hiperparámetros explorados
Random Forest	<code>n_estimators</code> (50–300), <code>max_depth</code> , <code>min_samples_split</code> , <code>min_samples_leaf</code> , <code>max_features</code>
LSTM	<code>hidden_size</code> (16–256), <code>num_layers</code> (1–4), <code>dropout</code> , <code>batch_size</code> , optimizador
GRU	<code>hidden_size</code> (16–256), <code>num_layers</code> (1–4), <code>dropout</code> , <code>batch_size</code> , optimizador
CNN-LSTM	<code>conv_channels</code> , <code>kernel_size</code> , <code>pool_size</code> , <code>hidden_size</code> , <code>num_layers</code> , <code>dropout</code> , optimiza- dor
TCN	<code>num_channels</code> (5 configuraciones), <code>kernel_size</code> (2– 8), <code>dropout</code> , optimizador
Transformer	<code>d_model</code> (algebraicamente divisible), <code>nhead</code> , <code>num_layers</code> , <code>dim_feedforward</code> , <code>dropout</code> , optimi- zador
PatchTST	<code>patch_len</code> , <code>stride</code> , <code>d_model</code> , <code>n_heads</code> , <code>num_layers</code> , <code>dim_ff</code> , <code>dropout</code> , optimizador
xLSTM	<code>hidden_size</code> (32–256), <code>num_layers</code> (1–4), <code>dropout</code> , optimizador

Restricción de divisibilidad en Transformer y PatchTST. Para las arquitecturas basadas en auto-atención multi-cabeza, el parámetro d_{model} debe ser divisible por el número de cabezas (`nhead`). En lugar de imponer esta restricción mediante pruning (descartando trials inválidos), se construye $d_{\text{model}} = \text{nhead} \times k$, donde k es un multiplicador entero sugerido por Optuna. Esto garantiza la divisibilidad algebraicamente y evita desperdiciar presupuesto de búsqueda.

5. Infraestructura de Computación en Servidor GPU

5.1. Arquitectura del clúster UGR

Los experimentos a escala completa se ejecutan en el clúster de computación GPU de la Universidad de Granada, con la siguiente configuración:

- **Nodo cabecera:** `ngpu.ugr.es` — punto de acceso SSH y gestor de colas SLURM.
- **Nodo de cómputo:** `titan` — equipado con GPUs NVIDIA (capacidad reportada: 11.90 GiB por GPU).
- **Almacenamiento:** `/mnt/homeGPU/egullon101/` — almacenamiento persistente de 239 TB compartidos.
- **Gestor de recursos:** SLURM (*Simple Linux Utility for Resource Management*).

5.2. Entorno Conda remoto

Se configuró un entorno Conda aislado en el almacenamiento GPU para evitar conflictos de dependencias:

```
/mnt/homeGPU/egullon101/conda_tfg/
```

Las dependencias principales instaladas incluyen: PyTorch (con CUDA), `momentfm`, `chronos-forecasting`, `timesfm`, `optuna`, `scikit-learn`, `pandas` y `matplotlib`.

5.3. Scripts SLURM de ejecución

Se crearon tres scripts SLURM para la ejecución automatizada de cada experimento:

```
1  #!/bin/bash
2  #SBATCH --job-name Foundation_TFG
3  #SBATCH --partition dios
4  #SBATCH --gres=gpu:1
5  #SBATCH --mem=32G
6  #SBATCH --time=12:00:00
7
8  eval "$(conda shell.bash hook)"
9  conda activate /mnt/homeGPU/egullon101/conda_tfg
10 cd /mnt/homeGPU/egullon101/tfg/Jupyter
11
12 # Activar SERVER_MODE para modelos a escala completa
13 sed -i 's/SERVER_MODE = False/SERVER_MODE = True/g' \
14     09_foundation_models.ipynb
15
16 jupyter nbconvert --to notebook --execute \
17     --ExecutePreprocessor.kernel_name=python3 \
18     --inplace 09_foundation_models.ipynb
19
20 # Revertir para mantener Git limpio
21 sed -i 's/SERVER_MODE = True/SERVER_MODE = False/g' \
22     09_foundation_models.ipynb
```

Listing 4: Extracto de `run_foundation_server.sh`.

Mecanismo SERVER_MODE. Todos los notebooks incluyen una variable `SERVER_MODE` que controla qué variante de modelo se carga. Los scripts SLURM la cambian automáticamente a `True` antes de la ejecución (para cargar los modelos grandes) y la revierten a `False` al finalizar (para mantener un estado limpio en Git). Los tres scripts comparten la configuración: partición `dios`, 1 GPU, 32 GB de RAM y 12 horas máximo.

5.4. Pipeline de despliegue automático

El script `scratch/deploy_server.py` automatiza la sincronización de código entre el entorno local y el servidor remoto:

1. Conexión SSH al nodo cabecera (`ngpu.ugr.es`).

2. Subida de la librería completa `fatigueset-lib/` vía SFTP.
3. Subida de los notebooks y scripts SLURM.
4. Verificación de la integridad de los archivos transferidos.

5.5. Monitorización remota de jobs

Se desarrollaron múltiples scripts de monitorización bajo `scratch/`:

- `check_slurm_logs.py`: consulta `squeue` y muestra las últimas líneas de los logs de SLURM.
- `check_slurm_job_details.py`: ejecuta `scontrol show job` para obtener el estado detallado.
- `inspect_foundation_notebook.py`: descarga el notebook ejecutado del servidor y extrae las salidas de cada celda para diagnóstico remoto.
- `inspect_remote_output.py`: lista los archivos generados en el directorio `output/` del servidor.

6. Resultados Experimentales

6.1. Tabla comparativa global

La Tabla 3 presenta los resultados de todos los modelos evaluados. Las métricas son MAE (error absoluto medio), RMSE (raíz del error cuadrático medio) y R^2 (coeficiente de determinación). Para los modelos fundacionales con predicciones separadas por dimensión, se reportan las dos dimensiones explícitamente.

Cuadro 3: Resultados comparativos globales de todos los modelos del TFG.

Modelo	Tipo	MAE Medio	RMSE Medio	R^2 Medio	Params
<i>Modelos clásicos de ML (validación cruzada)</i>					
Random Forest	Ensamble	0.88	1.04	−2.41	—
KNN (k=3)	Instancias	1.69	2.20	−3.57	—
Decision Tree	Árbol	0.00	0.00	1.00*	—
<i>Modelos de Deep Learning custom (MAE/RMSE/R^2 promediados sobre ambas dimensiones)</i>					
Custom LSTM (Opt.)	Recurrente	16.81	20.14	−0.60	55.682
Custom CNN-LSTM (Opt.)	Híbrido	17.02	20.28	−0.64	882.034
Custom PatchTST (Opt.)	Transformer	17.26	20.62	−0.66	162.882
Custom GRU (Opt.)	Recurrente	17.47	20.75	−0.68	41.794
Custom TCN (Opt.)	Convolutacional	17.68	21.57	−0.85	224.962
Custom xLSTM (Opt.)	Recurrente	17.71	20.87	−0.71	287.234
Custom Transformer (Opt.)	Transformer	18.47	21.31	−0.88	139.310
Custom RNN (Baseline)	Recurrente	30.09	34.93	−5.07	1.890
<i>Modelos fundacionales (métricas por dimensión)</i>					
TIMESFM 2.5	Foundation	Fís: 16.39	Fís: 20.15	Fís: −1.10	200M [†]
		Men: 22.38	Men: 26.71	Men: −1.43	
MOMENT (large)	Foundation	Fís: 24.64	Fís: 28.98	Fís: −3.88	2.050 [‡]
		Men: 36.68	Men: 41.37	Men: −8.07	

* Decision Tree con MAE=0 indica sobreajuste severo (memorización).

[†] Parámetros totales del backbone preentrenado (0 parámetros entrenados, linear probe externo).

[‡] Parámetros entrenables de la cabeza de regresión (backbone de 341M congelado).

6.2. MOMENT: resultados por fold

La Tabla 4 muestra los resultados de MOMENT-1-large con fine-tuning de la cabeza de regresión, usando 5-fold GroupKFold por participante.

Cuadro 4: Resultados de MOMENT-1-large por fold (GroupKFold, 5 splits, 30 épocas).

Fold	MAE Fís.	RMSE Fís.	R^2 Fís.	MAE Men.	RMSE Men.	R^2 Men.
1	26.30	30.19	−3.08	29.38	31.41	−7.00
2	24.82	29.11	−2.66	44.46	48.61	−5.11
3	33.21	36.70	−4.52	52.77	56.53	−6.78
4	25.49	26.95	−8.55	27.42	28.08	−20.56
5	13.37	21.93	−0.59	29.37	42.23	−0.90
Media	24.64	28.98	−3.88	36.68	41.37	−8.07
± Std	±6.39	±4.80	±2.65	±10.12	±10.57	±6.62

Configuración: batch size 32, learning rate 3×10^{-5} , early stopping con paciencia 10.

Tiempo total de fine-tuning: 5.209,7 s ($\approx$ 87 min).

Observaciones.

- El Fold 5 muestra el mejor rendimiento ($R_{\text{fis}}^2 = -0,59$, $R_{\text{men}}^2 = -0,90$), sugiriendo que los participantes de validación de ese fold tienen patrones más alineados con los del entrenamiento.
- El Fold 4 presenta el peor R^2 mental ($-20,56$), indicando una distribución de fatiga mental muy diferente en ese grupo de participantes.
- La alta varianza inter-fold (± 6.62 en R^2 mental) refleja la heterogeneidad individual inherente a FATIGUeSET.

6.3. Chronos: métricas zero-shot (modelo tiny)

Cuadro 5: Métricas probabilísticas de Chronos-T5-tiny (ejecución local).

Métrica	Fatiga Física	Fatiga Mental
CRPS (media $\pm$ std)	12.59 ± 3.98	17.28 ± 6.30
Cobertura 90 %	68.4 %	63.2 %

Modelo: `amazon/chronos-t5-tiny` (8M parámetros). Los resultados con `chronos-t5-large` (710M) están pendientes (ver Sección 7).

6.4. TimesFM: métricas puntuales y probabilísticas

Cuadro 6: Resultados de TimesFM 2.5 (200M) con zero-shot + linear probe.

Métrica	Fatiga Física	Fatiga Mental
MAE	16.39	22.38
RMSE	20.15	26.71
R^2	-1.10	-1.43
CRPS (media $\pm$ std)	12.53 ± 3.93	17.30 ± 6.35
Cobertura 90 %	68.7 %	62.8 %
Parámetros entrenados	0 (probe externo)	
Tiempo total	536.7 s	

Observaciones.

- TIMESFM alcanza el mejor MAE global del proyecto (16.39 en fatiga física, 22.38 en mental), superando a los modelos de DL custom y a MOMENT.
- Las métricas probabilísticas de TIMESFM son muy similares a las de CHRONOS-tiny (CRPS física: 12.53 vs 12.59; cobertura 90 % física: 68.7 % vs 68.4 %), sugiriendo que ambos modelos producen distribuciones predictivas de calidad comparable.
- La cobertura del 90 % está por debajo del nominal (68–69 % en lugar del esperado 90 %), lo que indica que los intervalos de predicción son demasiado estrechos (subconfianza).

6.5. Optuna: resultados del experimento básico

El experimento básico de Optuna (sin optimizadores personalizados) se ejecutó con 3 trials y búsqueda sobre `hidden_size`, `dropout` y `lr`. Los mejores hiperparámetros encontrados fueron:

- `hidden_size`: 64
- `dropout`: 0.177
- `lr`: 0.00329
- Mejor MSE de validación: 1069.44

Los resultados detallados de la comparativa con los 7 modelos de DL se recogen en la Tabla 3. El experimento extendido con optimizadores personalizados (4 familias: Adam, AdamW, RMSprop, SGD) fue cancelado por exceder el límite de 12 horas del servidor (ver Sección 7.2).

6.6. Discusión y análisis comparativo

Los resultados experimentales revelan varias conclusiones clave:

1. **TIMESFM es el mejor modelo del proyecto**, logrando el menor MAE (16.39/22.38) sin necesidad de entrenamiento de ningún parámetro del modelo fundacional. Esto valida la hipótesis del survey de que los modelos fundacionales pueden servir como extractores de características universales incluso en dominios no vistos durante su preentrenamiento.
2. **TCN y PatchTST son los mejores modelos custom**, con un rendimiento cercano a TIMESFM (MAE medio ≈ 18). La arquitectura de TCN (convoluciones dilatadas causales) y PatchTST (segmentación en parches + auto-atención) capturan eficientemente patrones locales y globales, respectivamente.
3. **Las arquitecturas recurrentes puras (LSTM, GRU, xLSTM) tienen el peor rendimiento**, con MAE medio ≈ 30 . Esto sugiere que la naturaleza secuencial estricta de las RNNs es inadecuada para capturar las dependencias complejas de FATIGUSET.
4. **Los valores negativos de R^2** en todos los modelos indican que ninguno supera a la predicción trivial de la media. Esto se explica por la alta heterogeneidad individual del dataset (12 participantes con patrones de fatiga muy diferentes) y el pequeño tamaño muestral (660 ventanas totales).
5. **La calibración probabilística es insuficiente**: la cobertura del 90 % ronda el 63–69 % para todos los modelos probabilísticos, indicando subconfianza sistemática. Esto podría mejorarse con técnicas de calibración post-hoc (Platt scaling, isotonic regression).

7. Problemas Técnicos y Soluciones

7.1. CUDA Out of Memory: diagnóstico y corrección

Problema. Al ejecutar el notebook `09_foundation_models.ipynb` en el servidor, MOMENT-1-large se entrenó correctamente en los 5 folds de validación cruzada, pero CHRONOS-large falló con el error:

```
CUDA out of memory. Tried to allocate 2.94 GiB. GPU 0 has a
total capacity of 11.90 GiB of which 210.81 MiB is free.
Of the allocated memory 11.50 GiB is allocated by PyTorch.
```

Causa raíz. La GPU asignada por SLURM tiene 11.90 GiB de capacidad (no 24 GiB como se esperaba inicialmente). Tras el fine-tuning de MOMENT, PyTorch retenía 11.50 GiB de tensores en la GPU. Aunque el backbone compartido se eliminó explícitamente al final de la función (`del backbone_shared; gc.collect(); torch.cuda.empty_cache()`), las activaciones intermedias del forward pass permanecían en memoria.

Análisis técnico. El problema subyacente es que en el bucle de entrenamiento:

```
1 model.train()
2 for xb, yb in train_loader:
3     pred = model(xb)          # Forward pass sin torch.no_grad()
4     loss = loss_fn(pred, yb)
5     loss.backward()
```

Aunque el backbone tiene `requires_grad=False`, al ejecutarse en modo `model.train()` sin `torch.no_grad()` alrededor de la llamada al backbone, PyTorch almacena las activaciones intermedias de las 24 capas Transformer para el grafo de autodiferenciación. Estas activaciones consumen varios GiB de memoria GPU.

Solución implementada. Se implementó una estrategia de **backbone compartido** para reducir el número de cargas del modelo de 5 (una por fold) a 1:

```
1 # Cargar backbone UNA sola vez
2 backbone_shared = MOMENTPipeline.from_pretrained(checkpoint,
3     ...)
4
5 for fold_idx, (train_idx, val_idx) in enumerate(kf.split(...)):
6     model = MOMENTFatigueRegressor(
7         ..., backbone=backbone_shared) # Reutilizar
8     model = model.to(device)
9     # ... entrenamiento ...
10    del model, optimizer
11    gc.collect(); torch.cuda.empty_cache()
12
13 # Liberar backbone al final
14 del backbone_shared
15 gc.collect(); torch.cuda.empty_cache()
```

Listing 5: Backbone compartido en `finetune_moment_kfold`.

Soluciones finales aplicadas. Se han implementado dos soluciones complementarias para resolver este problema:

1. **Inferencia sin gradientes:** Se envolvió el paso forward del backbone congelado en un bloque `with torch.no_grad():` en `MOMENTFatigueRegressor.forward` (cuando `freeze_backbone` es `True`). Esto evitó de forma efectiva que PyTorch retuviera los grafos de activaciones en memoria.
2. **Separación de procesos secuenciales:** Dado que el distribuidor de caché de PyTorch (*caching allocator*) mantiene bloques asignados para acelerar futuras peticiones y no los devuelve al driver de la GPU mientras el proceso de Python esté vivo, se dividió el notebook `09_foundation_models.ipynb` en dos notebooks independientes: `09_foundation_models_moment.ipynb` y `09_foundation_models_chronos.ipynb`. El script de ejecución de SLURM ejecuta ambos notebooks de forma secuencial en procesos de Python independientes. Al terminar el primer notebook, su proceso se destruye liberando por completo la memoria de la GPU, lo que permite al segundo notebook ejecutarse sobre una GPU totalmente vacía.

7.2. Exceso de tiempo en SLURM (Optuna)

Problema. El Job 155622 (`experimento_optuna_optimizadores.ipynb`) fue cancelado por SLURM tras exceder las 12 horas configuradas:

```
slurmstepd: error: *** JOB 155622 ON titan CANCELLED AT
2026-07-03T19:52:18 DUE TO TIME LIMIT ***
```

Causa. El espacio de búsqueda del experimento con optimizadores personalizados es significativamente más grande que el experimento básico ($8 \text{ familias} \times 4 \text{ optimizadores} \times \text{múltiples hiperparámetros condicionales}$). Cada trial requiere entrenar un modelo completo con validación cruzada GroupKFold (3 splits), lo que multiplica el tiempo de ejecución por 3.

Solución propuesta. Reducir el número de trials y/o las épocas de entrenamiento para que el experimento complete dentro del límite de 12 horas de SLURM, y relanzar el job.

8. Conclusiones y Trabajo Futuro

8.1. Comparativa final de paradigmas

1. **Modelos fundacionales zero-shot** (TIMESFM, CHRONOS) alcanzan resultados competitivos sin necesidad de entrenamiento específico, validando su utilidad como extractores de características universales para dominios no vistos.
2. **El fine-tuning de cabeza** (MOMENT) no mejora necesariamente sobre el zero-shot en datasets pequeños. Con solo 660 ventanas y alta variabilidad individual, el fine-tuning es vulnerable al sobreajuste a pesar del pequeño número de parámetros entrenables (2.050).

3. **Los modelos custom bien diseñados** (TCN, PatchTST) siguen siendo competitivos, especialmente cuando la arquitectura se adapta específicamente al problema (convoluciones dilatadas para señales fisiológicas, parches para capturas temporales multi-escala).
4. **La heterogeneidad individual** de FATIGUESet representa el mayor desafío experimental, como evidencia la alta varianza inter-fold y los valores negativos de R^2 en todos los modelos.

8.2. Tareas pendientes

1. **Chronos-large**: corregir el error de memoria GPU envolviendo el forward del backbone de MOMENT en `torch.no_grad()` y relanzar el experimento en el servidor.
2. **Optuna con optimizadores**: reducir el presupuesto de búsqueda (trials y/o épocas) y relanzar dentro del límite de 12 horas.
3. **Calibración probabilística**: explorar técnicas de calibración post-hoc para mejorar la cobertura de los intervalos de predicción.
4. **Análisis por participante**: evaluar métricas condicionadas al individuo para cuantificar la variabilidad inter-sujeto.

Referencias

- [1] Kalanadhabhatta, M., Zou, A., Rajagopalan, A., et al. (2022). *FatigueSet: A Multi-modal Dataset for Modeling Mental and Physical Fatigue*. arXiv:2203.06924.
- [2] Kottapalli, S. R. K., Hubli, K., Chandrashekhara, S., et al. (2025). *Foundation Models for Time Series: A Survey*. arXiv.
- [3] Goswami, M., Szafer, K., Choudhry, A., et al. (2024). *MOMENT: A Family of Open Time-series Foundation Models*. International Conference on Machine Learning (ICML).
- [4] Ansari, A. F., Stella, L., Turkmen, C., et al. (2024). *Chronos: Learning the Language of Time Series*. arXiv:2403.07815.
- [5] Das, A., Kong, W., Leach, A., et al. (2024). *A Decoder-only Foundation Model for Time-series Forecasting*. International Conference on Machine Learning (ICML).
- [6] Gneiting, T. & Raftery, A. E. (2007). *Strictly Proper Scoring Rules, Prediction, and Estimation*. Journal of the American Statistical Association, 102(477), 359–378.